

LogPulse

Application Log Analytics Dashboard

What's This Project About?

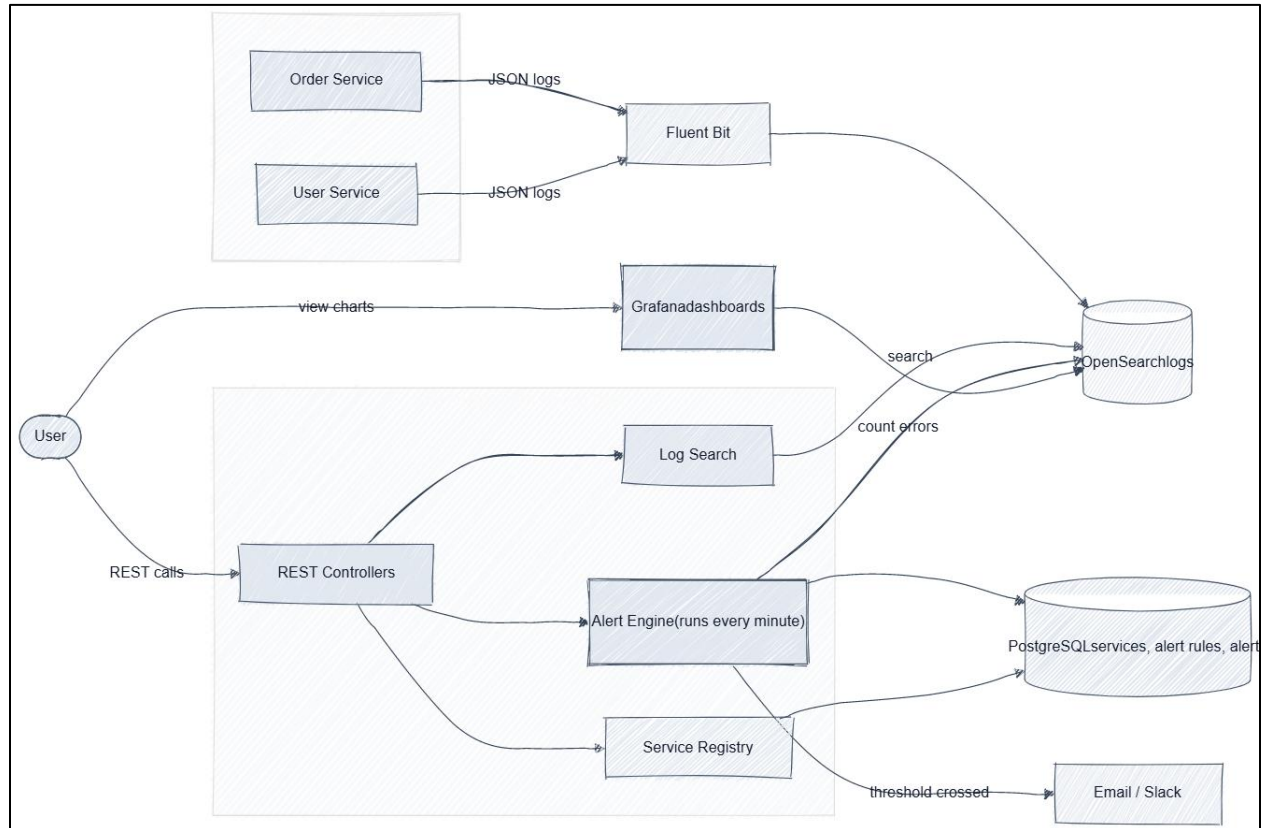
Right now, if a Java service throws a bunch of errors, someone has to notice it, SSH into a box, and grep through log files. LogPulse fixes that. It's a small, centralized log analytics setup that pulls logs from your Java services into one place, lets you search them instantly, shows you dashboards, and pings you on Slack or Email the moment something looks broken — all without babysitting anything, since it checks itself every minute.

In short: logs go in one door, you get search, dashboards, and alerts out the other. Built to run entirely in Docker, so it's one command to stand the whole thing up.

Core Stack

- ❑ Java 21 + Spring Boot 3 — the API and alert engine
- ❑ OpenSearch — stores and searches the logs
- ❑ Fluent Bit — ships logs from services to OpenSearch
- ❑ PostgreSQL — stores services, alert rules, and fired alerts
- ❑ Grafana — dashboards on top of the log data
- ❑ Docker Compose — runs the whole stack with one command

Architecture Diagram



Services log to Fluent Bit -> OpenSearch, with the API handling search, alerting, and service registration, and Grafana reading straight from OpenSearch.

Implementation Plan

PHASE 1

Foundation

Day 1 — Tooling & Project Setup

1. Install and configure Eclipse, Docker Desktop, PostgreSQL, Grafana, Java, Postman, and Git.
2. Create a basic Spring Boot application with a health-check endpoint.
3. Set up Docker Compose with PostgreSQL, OpenSearch, and Grafana services.
4. Publish the project to the repository and add setup instructions to the README.

Day 2 — Database Design

1. Create the **Service**, **Alert Rule**, and **Alert** tables in PostgreSQL.
2. Develop JPA entity classes for database mapping.
3. Create JPA repositories for basic CRUD operations.
4. Configure and test the PostgreSQL database connection.

PHASE 2

Core APIs & Log Pipeline

Day 3 — Service Registry API

1. Create APIs to register and view monitored services.
2. Add API key authentication to secure the endpoints.
3. Configure Swagger UI for API documentation and testing.
4. Verify that all APIs are accessible, secure, and properly documented.

Day 4 — Log Ingestion Pipeline

1. Configure applications to generate structured JSON logs.
2. Set up Fluent Bit to collect and forward logs.
3. Connect Fluent Bit to OpenSearch for log ingestion.
4. Verify that application logs appear in OpenSearch within a few seconds.

Day 5 — Log Search API

1. Create a log search API to retrieve logs from OpenSearch.
2. Enable filtering by service, log level, keyword, and time range.
3. Optimize the search queries for fast log retrieval.
4. Test and verify that log searches return accurate results quickly.

PHASE 3

Alerting

Day 6 — Alert Rules & Scheduler

1. Create APIs to configure alert rules with log level, threshold, and time window settings.
2. Store and manage alert rule configurations in the database.
3. Implement a scheduler to check all active alert rules every minute.
4. Trigger alerts automatically when the configured thresholds are exceeded.

Day 7 — Alert Engine

1. Implement a scheduler to monitor logs and evaluate alert rules.
2. Trigger alerts automatically when rule thresholds are exceeded.
3. Create APIs to view active and historical alerts.

4. Add an API to resolve alerts after they have been addressed.

Day 8 — Notifications

1. Integrate Slack and Email notifications for alert delivery.
2. Configure alert rules and thresholds for error monitoring.
3. Test alert generation by triggering errors that exceed the defined threshold.
4. Verify that Slack and Email alerts are received within one minute of the issue occurring.

PHASE 4

Polish & Ship

Day 9 — Dashboards, Security & Tests

1. Create Grafana dashboards to visualize log and monitoring data.
2. Ensure API key authentication is enforced across all endpoints.
3. Increase unit test coverage to 70% or higher.
4. Create a Postman collection covering all application APIs.

Day 10 — Dockize & Deploy

1. Create a Docker Compose setup to start the entire application stack with a single command.
2. Deploy the application and ensure all services start successfully.
3. Run end-to-end smoke testing to verify key functionalities.
4. Update the documentation and prepare the solution for demonstration.